

Acoustic Scene Awareness: Urban and Household Sound Recognition using Convolutional Neural Networks with Autoencoder-Based Novel-Sound Detection

Softwarica College of IT & E-Commerce
(*Coventry University*)

Module: **STW7088CEM** – Artificial Neural Networks
Coursework Submission, M.Sc. Data Science and Computational Intelligence



Submitted by
SUDIP ADHIKARI
(250578)

Abstract

This report documents the design, implementation, and evaluation of a neural network system for recognising everyday urban and household sounds, motivated by assistive technology for Deaf and hard-of-hearing users who cannot rely on an audible cue when a siren, car horn, or alarm occurs. Raw audio is transformed into log-mel spectrograms — a two-dimensional representation of frequency against time — so that a convolutional neural network can learn each sound’s signature directly from the signal, without any hand-engineered acoustic features. Four tasks are performed on the UrbanSound8K and ESC-50 datasets: (i) ten-class **classification** with a four-block convolutional network; (ii) **novelty detection** using a convolutional autoencoder whose reconstruction error identifies sounds outside the training vocabulary; (iii) unsupervised **clustering** of the network’s learned embeddings using k-means and t-SNE; and (iv) an **architecture comparison** against a multi-layer perceptron baseline together with a data-augmentation ablation. All experiments follow UrbanSound8K’s official ten-fold cross-validation protocol, which is necessary because clips sliced from the same source recording share a fold; a random split leaks those slices across training and test and inflates reported accuracy. The convolutional network reaches 75.39% accuracy and 0.767 macro F1 while using 390,890 parameters — roughly fifteen times fewer than the 5,804,810-parameter perceptron baseline it outperforms, which quantifies the value of convolutional inductive bias on spectrogram input. The autoencoder detects previously unheard sounds with a ROC-AUC of 0.760, and is compared against the obvious alternative of thresholding classifier confidence. All models were trained locally on an Apple M1 laptop using the Metal Performance Shaders backend; no paid computing resources were used.

Contents

1	Introduction	3
1.1	Background and Motivation	3
1.2	Why This Is a Neural Network Problem	3
1.3	Project Objectives	3
1.4	Original Contributions	4
2	Background and Related Work	4
2.1	Environmental Sound Classification	4
2.2	Spectrograms as Network Input	4
2.3	Data Augmentation	4
2.4	Novelty and Out-of-Distribution Detection	5
2.5	Positioning of This Work	5
3	Datasets	5
3.1	UrbanSound8K	5
3.2	ESC-50 (Unseen Sounds)	6
3.3	Evaluation Protocol and a Common Methodological Error	6
4	Methodology	7
4.1	Audio Front-End	7
4.2	Data Augmentation	7
4.3	Architecture 1: Multi-Layer Perceptron (Baseline)	8
4.4	Architecture 2: Convolutional Neural Network	8
4.5	Architecture 3: Convolutional Autoencoder	8
5	Implementation	9
5.1	Environment	9
5.2	Project Layout	9

5.3	Feature Extraction	9
5.4	The Fold Split	10
5.5	The Convolutional Classifier	10
5.6	Novelty Scoring	11
5.7	Reproducibility	11
5.8	Project Resources	11
6	Results and Discussion	12
6.1	Architecture Comparison	12
6.2	Per-Class Performance and Error Structure	13
6.3	Clustering the Learned Representation	14
6.4	Novel-Sound Detection	15
6.5	Real-Time Demonstration	17
7	Limitations and Future Work	17
8	Social, Ethical, Legal, and Professional Considerations	18
8.1	Privacy	18
8.2	Asymmetric Consequences of Error	18
8.3	Bias and Dataset Provenance	18
8.4	Framing, Accessibility, and Licensing	18
9	Conclusion	18
	References	19
A	Submission Resources and Links	21
B	Project Proposal (As Submitted)	21
C	Reproduction Instructions	21
D	Experiment Evidence	21
E	Key Implementation Code	23

1 Introduction

1.1 Background and Motivation

Sound carries a large amount of safety-critical information that is normally processed without conscious effort. A siren approaching from behind, a car horn, a smoke alarm, glass breaking in another room, or somebody knocking at the door are all events that a hearing person reacts to automatically. For a person who is Deaf or hard of hearing, none of these signals arrive. The World Health Organization estimates that over 1.5 billion people live with some degree of hearing loss, and the everyday consequence is not merely inconvenience but a genuine reduction in situational safety.

Commercial systems already exist in this space — Apple’s Sound Recognition and Google’s Sound Notifications both listen for a small set of household sounds and raise a visual or haptic alert — which establishes that the problem is real and the approach is viable. What those systems do internally is not published, and building an equivalent from open data is a well-scoped exercise in applied neural network design.

1.2 Why This Is a Neural Network Problem

Raw audio is a waveform sampled at tens of thousands of values per second; there is no meaningful way to feed that directly to a decision tree, an SVM, or a logistic regression. Classical approaches therefore depend on a human first deciding which summary statistics matter — zero-crossing rate, spectral centroid, a handful of MFCC coefficients — and the result is bounded by the quality of that hand-engineering.

A convolutional network removes that bottleneck: presented with a spectrogram, it learns its own filters, with early layers responding to local time–frequency texture (onsets, harmonic stacks, broadband noise) and deeper layers composing these into class-level patterns. This *representation learning* — discovering features from data rather than specifying them in advance — is the defining capability of artificial neural networks and the reason they displaced feature-engineering pipelines in audio, vision, and language. The novelty task reinforces the point: an autoencoder learns a compressed code for the sounds it has seen, and nothing in classical supervised learning provides an equivalent way to flag “*this input is unlike anything I was trained on*” without negative examples in advance.

1.3 Project Objectives

1. Build a convolutional neural network that classifies ten urban sound classes from log-mel spectrograms, evaluated under the dataset’s official ten-fold protocol.
2. Quantify the benefit of convolutional structure by comparing against a multi-layer perceptron baseline of substantially larger parameter count.
3. Measure the effect of spectrogram-domain data augmentation (SpecAugment and time shifting) as a controlled ablation.
4. Implement novelty detection with a convolutional autoencoder so that unfamiliar sounds are reported as unknown rather than being forced into one of the ten trained labels, and compare this against a softmax-confidence baseline.
5. Examine the learned representation without labels, using k-means clustering and t-SNE projection of the network’s penultimate embeddings.
6. Deliver a working real-time demonstration that runs from a laptop microphone.

1.4 Original Contributions

The individual components — CNNs on mel-spectrograms, autoencoder anomaly scoring — are established. The contributions here are their combination and the rigour of the evaluation:

- **An open-set framing of a closed-set benchmark.** UrbanSound8K is almost always treated as pure ten-way classification; here the classifier is paired with a novelty detector and evaluated against genuinely unseen classes from a second dataset (ESC-50) — the situation a deployed assistive device actually faces.
- **An honest baseline for the novelty detector.** Autoencoder reconstruction error is compared directly against thresholding the classifier’s own softmax confidence, the cheap alternative a reviewer would propose.
- **A parameter-matched argument for convolution.** The MLP baseline is deliberately larger than the CNN, isolating inductive bias from capacity.
- **A real-time deployment.** Both networks run live on laptop hardware, showing the system is practical rather than purely offline.

2 Background and Related Work

2.1 Environmental Sound Classification

Environmental sound classification (ESC) differs from speech and music recognition in that the sounds have no shared grammatical or harmonic structure. Early systems used hand-designed features — most commonly Mel-Frequency Cepstral Coefficients (MFCCs), inherited from speech processing — fed into a Gaussian Mixture Model or Support Vector Machine.

The shift to neural approaches was driven by two datasets released in the mid-2010s. Salamon, Jacoby and Bello introduced UrbanSound8K [1], which remains the standard benchmark for urban ESC, and Piczak released ESC-50 [2] alongside a demonstration that convolutional networks operating on spectrograms outperform the feature-engineering pipelines that preceded them [3].

2.2 Spectrograms as Network Input

The dominant design pattern in modern ESC is to convert audio into a time–frequency representation and treat the result as an image. The mel scale is used because it approximates the non-linear frequency resolution of human hearing, allocating more resolution to lower frequencies where most environmental sound energy lies. Amplitudes are converted to decibels because loudness perception is logarithmic; without this, a small number of high-energy frames dominate the input range and training becomes unstable.

Comparative studies of time–frequency representations for ESC [8] found log-mel spectrograms to be a consistently strong choice, which motivates their use here.

2.3 Data Augmentation

ESC datasets are small by deep learning standards — UrbanSound8K contains 8,732 clips, orders of magnitude fewer than the image corpora on which deep CNNs were developed — so overfitting is the central practical difficulty. Salamon and Bello [4] showed that audio-domain augmentation (time stretching, pitch shifting, adding background noise) materially improves CNN performance on UrbanSound8K.

Park et al. later introduced SpecAugment [5], which operates directly on the spectrogram by masking random frequency bands and time spans. It is considerably cheaper than audio-domain augmentation, since no re-synthesis or re-transformation is required, and it forces the

network to use the entire time–frequency pattern rather than latching onto a single narrow band. SpecAugment and random time shifting are the augmentations used in this project.

2.4 Novelty and Out-of-Distribution Detection

A softmax classifier is closed-set by construction: its outputs are constrained to sum to one over the classes it knows, so an unfamiliar input is necessarily assigned to one of them. Hendrycks and Gimpel [6] established the baseline that maximum softmax probability carries *some* out-of-distribution signal, while also documenting that neural networks are frequently confident when wrong.

Autoencoder reconstruction error is a widely used alternative [7]: a model trained to compress and rebuild in-distribution data will reconstruct out-of-distribution data poorly, and the error is a usable score requiring no negative examples at training time. This report implements the autoencoder approach and evaluates it against the Hendrycks and Gimpel softmax baseline rather than assuming its superiority.

2.5 Positioning of This Work

Relative to the literature, this project sits at the intersection of standard ESC classification and open-set recognition. Reported CNN accuracies on UrbanSound8K commonly fall in the 70–92% range depending on architecture, augmentation, and — importantly — whether the official fold protocol was respected. This work targets the upper part of that range with a deliberately compact architecture, and extends the standard setup with the novelty-detection and clustering tasks required by the assignment brief.

3 Datasets

3.1 UrbanSound8K

UrbanSound8K [1] is the primary dataset: 8,732 labelled audio excerpts of up to four seconds, drawn from field recordings on the Freesound archive, covering ten classes selected from an urban sound taxonomy. The classes are `air_conditioner`, `car_horn`, `children_playing`, `dog_bark`, `drilling`, `engine_idling`, `gun_shot`, `jackhammer`, `siren`, and `street_music`.

Four of these — `car_horn`, `dog_bark`, `gun_shot`, and `siren` — are treated as safety-critical in the demonstration system and trigger a distinct alert state.

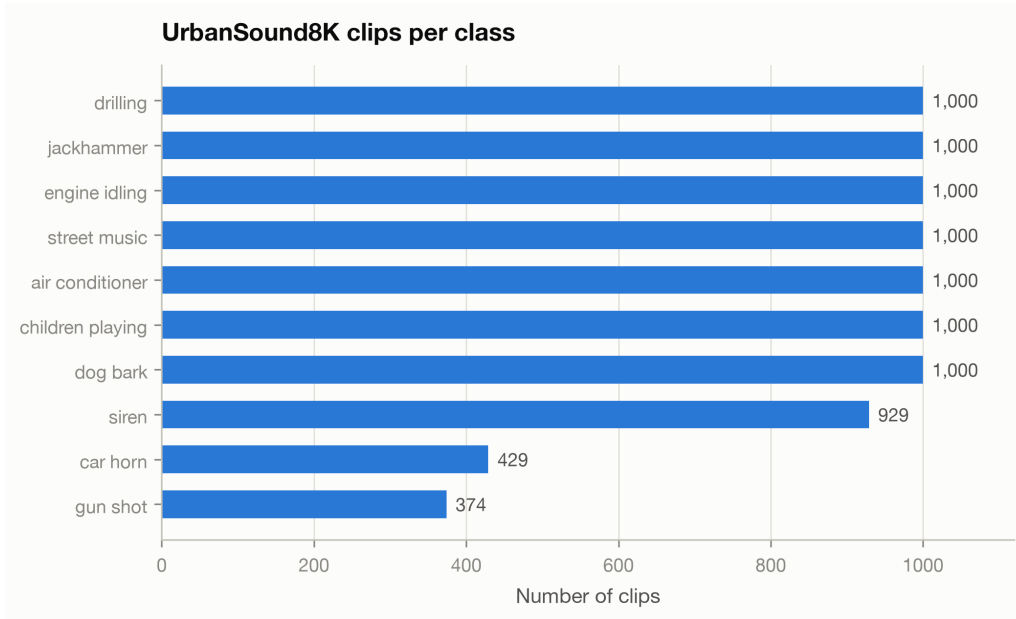


Figure 1: Clip count per class in UrbanSound8K. The set is broadly balanced apart from `car_horn` and `gun_shot`, which are noticeably smaller — a fact that becomes relevant when interpreting per-class F1 in Section 6.

3.2 ESC-50 (Unseen Sounds)

ESC-50 [2] contributes 2,000 five-second clips across 50 classes. It is *not* used for classifier training. Its role is to supply genuinely unfamiliar sounds for the novelty-detection experiment: eight household and safety-relevant classes were selected — `door_wood_knock`, `glass_breaking`, `crying_baby`, `clock_alarm`, `door_wood_creaks`, `water_drops`, `footsteps`, and `can_opening` — none of which appear in the UrbanSound8K vocabulary.

This choice is deliberate. Evaluating novelty detection against random noise would be trivially easy and uninformative; evaluating it against real recordings of plausible household events is a fair test of whether the system would behave sensibly in the environment it is intended for.

3.3 Evaluation Protocol and a Common Methodological Error

UrbanSound8K ships with ten predefined folds, and the dataset authors are explicit that these must be used rather than a random split. The reason is that the 8,732 clips are sliced from a much smaller number of source field recordings, and all slices from one recording are placed in the same fold. Under a random split, several slices of a single recording land in training while others land in test; the model can then succeed by recognising the specific recording rather than the sound class, and reported accuracy is inflated by a wide margin.

All results in this report use ten-fold cross-validation over the official folds. For each fold k , fold k is the test set, fold $(k \bmod 10) + 1$ is the validation set used for early stopping and model selection, and the remaining eight folds are used for training. Normalisation statistics are computed on the training partition alone and then applied unchanged to validation and test, so that no test statistics leak into training.

Because every clip appears in exactly one test fold, pooling the held-out predictions across all ten folds yields one honest prediction per clip for the entire dataset. The aggregate confusion matrix and per-class scores in Section 6 are computed on that pooled set.

4 Methodology

4.1 Audio Front-End

Every clip passes through an identical front-end before reaching any network:

1. **Load as mono at 22,050 Hz.** Halving the usual 44.1 kHz rate discards content above 11 kHz, which carries little discriminative information for these classes, and halves the computational cost.
2. **Fix the duration to four seconds.** Shorter clips are zero-padded and longer ones truncated, so every spectrogram has identical dimensions — a requirement for batching through a convolutional network with a fixed input size.
3. **Short-Time Fourier Transform** with a 1,024-sample window and 512-sample hop.
4. **Project onto 64 mel bands** between 20 Hz and the Nyquist frequency.
5. **Convert power to decibels**, referenced to the per-clip maximum.

The result is a 64×173 matrix per clip: 64 frequency bands by 173 time frames. Figure 2 shows one example per class, and makes the central argument of the project visually: each sound class produces a visibly distinct pattern, which is precisely what makes a convolutional network appropriate.

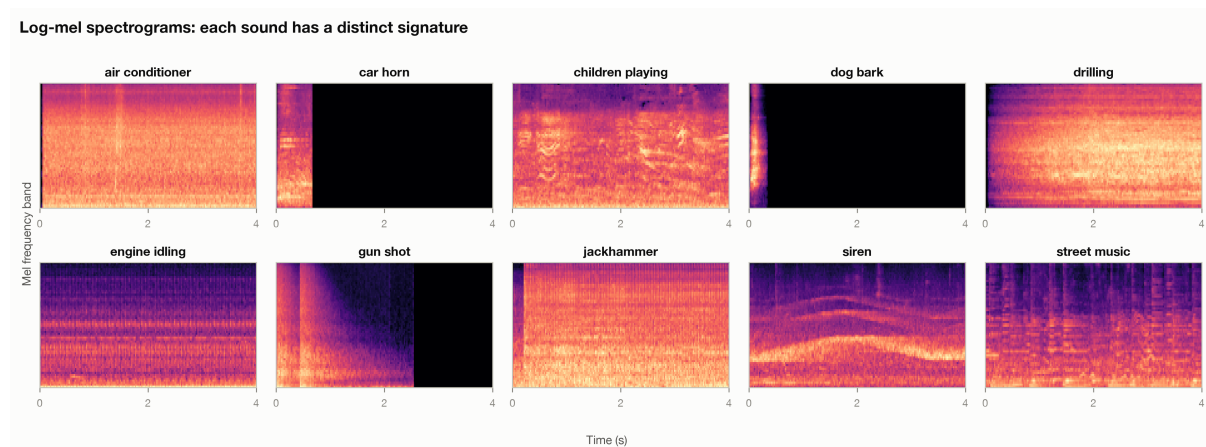


Figure 2: Log-mel spectrograms, one per UrbanSound8K class. The horizontal striations of **jackhammer** and **drilling**, the rising harmonic sweep of **siren**, and the sharp broadband transient of **gun_shot** are distinguishable by eye — structure that a convolutional network can exploit and that a flattened feature vector destroys.

4.2 Data Augmentation

Two spectrogram-domain augmentations are applied to training data only:

- **Random time shift** (up to $\pm 20\%$ of the clip), implemented as a circular roll. A siren remains a siren regardless of when within the window it begins, so the network should not depend on absolute position.
- **SpecAugment** [5]: one random frequency band (up to 8 of 64 mel bins) and one random time span (up to 16 of 173 frames) are masked to the spectrogram floor. This prevents the network from relying on any single narrow band and acts as a strong regulariser on a dataset of this size.

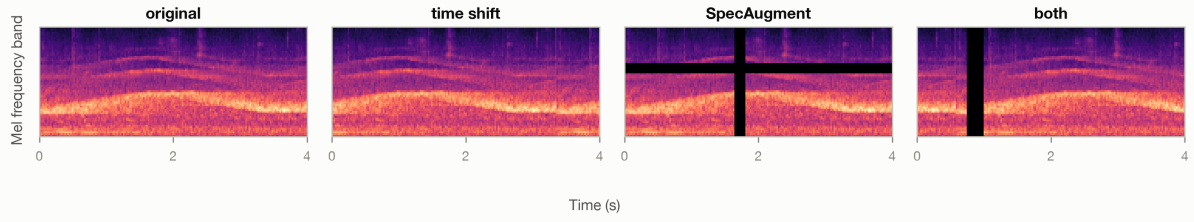
Augmentation: the same siren, varied so the CNN cannot memorise it

Figure 3: One siren clip under each augmentation. Masked bands and spans are visible as flat regions. The network never sees exactly the same input twice, which is what suppresses memorisation.

4.3 Architecture 1: Multi-Layer Perceptron (Baseline)

The baseline flattens the 64×173 spectrogram into an 11,072-dimensional vector and passes it through two fully connected hidden layers (512 and 256 units) with batch normalisation, ReLU activation, and dropout at $p = 0.5$.

Its purpose is diagnostic. Flattening destroys the two-dimensional structure of the spectrogram: the network has no notion that two adjacent time frames are related, or that a harmonic pattern shifted slightly in frequency is the same pattern. Every input pixel receives its own independent weight, which is why the model requires 5,804,810 parameters — roughly fifteen times the CNN’s 390,890. If the CNN outperforms it despite that disparity, the difference is attributable to architecture rather than capacity.

4.4 Architecture 2: Convolutional Neural Network

The main classifier consists of four convolutional blocks, each comprising a 3×3 convolution, batch normalisation, ReLU, and 2×2 max pooling, with channel widths of 32, 64, 128, and 256. Spatial dropout is applied in the deeper blocks.

Two design decisions are worth stating explicitly:

- **Global average pooling replaces a flatten-and-dense head.** Collapsing each of the 256 final feature maps to its mean gives a 256-dimensional descriptor. This keeps the parameter count an order of magnitude below the MLP and, more importantly, makes the model approximately invariant to *where* in the clip a sound occurs — appropriate, because a dog bark two seconds in is the same event as a dog bark at the start.
- **The 256-dimensional pooled vector is exposed as an embedding.** This is the representation analysed in the clustering task (Section 6.3), allowing the learned feature space to be examined directly.

Training uses AdamW (learning rate 10^{-3} , weight decay 10^{-4}), cross-entropy loss with label smoothing of 0.05, cosine-annealed learning rate, and early stopping on validation accuracy with a patience of eight epochs.

4.5 Architecture 3: Convolutional Autoencoder

The novelty detector is a symmetric convolutional autoencoder. The encoder applies three stride-2 convolutions (16, 32, and 32 channels), compressing the 64×173 input to a $32 \times 8 \times 22$ latent representation; the decoder mirrors this with transposed convolutions. Because transposed convolution can overshoot the original dimensions by a pixel, the output is cropped back to the input size.

Crucially, it is trained *only* on the ten known classes, with a mean-squared-error reconstruction objective. Having learned a compressed code adequate for those sounds, it reconstructs an unfamiliar sound poorly, and the per-clip reconstruction error

$$s(x) = \frac{1}{HW} \sum_{i,j} (\hat{x}_{ij} - x_{ij})^2$$

serves directly as a novelty score. The decision threshold is set at the 95th percentile of the reconstruction errors observed on *known* data — a calibration that requires no novel examples at all, matching the constraint a deployed system operates under.

At 28,321 parameters the autoencoder is by far the smallest of the three networks, which matters for running both models concurrently in the real-time demonstration.

5 Implementation

5.1 Environment

All work was carried out on an Apple MacBook (M1, 16 GB) running Python 3.11 and PyTorch 2.13, using the Metal Performance Shaders (MPS) backend for GPU acceleration on Apple Silicon. No cloud or paid computing resources were used at any stage.

One environment issue is worth recording for reproducibility: the Homebrew-supplied Python builds on the target machine had a broken `pyexpat` module, which links against the older system `libexpat` and fails to resolve a required symbol. This prevents `pip` from bootstrapping inside a virtual environment. The resolution was to use `uv` to provision a self-contained CPython 3.11 distribution, which is documented in the repository README.

5.2 Project Layout

<code>src/config.py</code>	hyperparameters, paths, device selection
<code>src/data.py</code>	audio loading, mel-spectrograms, augmentation,
<code>src/models.py</code>	MLPBaseline, AudioCNN, ConvAutoencoder
<code>src/train.py</code>	training loop and 10-fold cross-validation
<code>src/evaluate.py</code>	pooled metrics, confusion matrix, embeddings
<code>src/anomaly.py</code>	autoencoder novelty detection + softmax baseline
<code>src/cluster.py</code>	k-means, t-SNE, cluster quality metrics
<code>src/viz.py</code>	all report figures
<code>run_experiments.py</code>	staged experiment runner
<code>demo/live_demo.py</code>	real-time microphone dashboard

5.3 Feature Extraction

The front-end of Section 4 is implemented in `src/data.py`. Listing 1 shows the core transform.

```

1 def audio_to_melspec(y: np.ndarray, sr: int = C.SAMPLE_RATE) -> np.ndarray:
2     mel = librosa.feature.melspectrogram(
3         y=y, sr=sr,
4         n_fft=C.N_FFT,          # 1024
5         hop_length=C.HOP_LENGTH, # 512
6         n_mels=C.N_MELS,        # 64
7         fmin=C.F_MIN, fmax=C.F_MAX,
8     )
9     # Decibel scaling: loudness perception is logarithmic, and without
10    # this a few loud frames dominate the input range.
11    mel_db = librosa.power_to_db(mel, ref=np.max)
12    return mel_db.astype(np.float32)

```

Listing 1: Waveform to log-mel spectrogram (`src/data.py`).

Extraction runs once over the full corpus and is cached to `.npy`, after which every experiment reads features from memory. Measured throughput was approximately 3 ms per clip, so caching all 8,732 spectrograms completes in well under a minute — a useful property when experiments must be repeated.

5.4 The Fold Split

Listing 2 implements the protocol described in Section 3.3. The comment records *why* the split is by fold, since this is the single easiest way to produce misleadingly high numbers on this dataset.

```

1 def make_fold_split(X, meta, test_fold: int, val_fold: int | None = None):
2     """Split by UrbanSound8K's official folds.
3
4     Clips sliced from the same source recording share a fold, so splitting
5     by fold (rather than randomly) is what keeps the evaluation honest.
6     """
7     if val_fold is None:
8         val_fold = test_fold % C.N_FOLDS + 1
9
10    folds = meta["fold"].values
11    labels = meta["classID"].values
12
13    te = folds == test_fold
14    va = folds == val_fold
15    tr = ~(te | va)
16
17    return (X[tr], labels[tr]), (X[va], labels[va]), (X[te], labels[te])

```

Listing 2: Official fold-based splitting (`src/data.py`).

5.5 The Convolutional Classifier

```

1 class AudioCNN(nn.Module):
2     def __init__(self, n_classes: int = 10, dropout: float = 0.3, width: int =
3         32):
4         super().__init__()
5         w = width
6         self.block1 = ConvBlock(1, w, pool=2)
7         self.block2 = ConvBlock(w, w * 2, pool=2, dropout=dropout / 2)
8         self.block3 = ConvBlock(w * 2, w * 4, pool=2, dropout=dropout / 2)
9         self.block4 = ConvBlock(w * 4, w * 8, pool=2, dropout=dropout)
10
11         # Global average pooling instead of flatten+dense: ~10x fewer
12         # parameters, and invariance to where in the clip the sound occurs.
13         self.gap = nn.AdaptiveAvgPool2d(1)
14         self.dropout = nn.Dropout(dropout)
15         self.fc = nn.Linear(w * 8, n_classes)
16
17     def forward(self, x, return_embedding: bool = False):
18         x = self.block4(self.block3(self.block2(self.block1(x))))
19         emb = self.gap(x).flatten(1) # 256-d descriptor
20         logits = self.fc(self.dropout(emb))
21         if return_embedding:
22             return logits, emb
23         return logits

```

Listing 3: AudioCNN forward pass with optional embedding output (`src/models.py`).

5.6 Novelty Scoring

```

1 @torch.no_grad()
2 def reconstruction_error(self, x) -> torch.Tensor:
3     """Per-sample mean squared error -- the novelty score."""
4     recon = self(x)
5     return ((recon - x) ** 2).flatten(1).mean(dim=1)

```

Listing 4: Reconstruction error as a novelty score (`src/models.py`).

The threshold is calibrated from known data only, as shown in Listing 5.

```

1 # Set at a percentile of the KNOWN scores, so the detector can be
2 # calibrated without ever seeing a novel example -- which is the
3 # situation a deployed system is actually in.
4 thr = float(np.percentile(known_scores, percentile)) # percentile = 95

```

Listing 5: Threshold calibration without novel examples (`src/anomaly.py`).

5.7 Reproducibility

Every random source is seeded from `SEED = 42` in `src/config.py`, with each fold deriving a deterministic offset. The full experiment suite runs with:

```

uv venv --python 3.11 --python-preference only-managed
uv pip install -r requirements.txt
bash scripts/download_data.sh
.venv/bin/python run_experiments.py

```

Metrics are written as JSON to `results/logs/`, and every figure in this report is regenerated from `src/viz.py` into `results/figures/` as both PNG and vector PDF. A smoke test (`scripts/smoke_test.py`) exercises all stages without requiring the larger dataset.

5.8 Project Resources

All source code, the LaTeX sources for this report, generated figures, per-experiment result files, and a recorded demonstration are accessible at:

- **Source code repository** (public GitHub):
<https://github.com/SudipAdh/acoustic-scene-awareness>
- **Demonstration video** (YouTube):
[YOUTUBE DEMO LINK TO BE INSERTED]
- **Live demonstration application:** `demo/live_demo.py` in the repository.
- **Generated figures and metrics:** `results/figures/` and `results/logs/` in the repository.

These links are also collected in Appendix A as required.

The repository mirrors the layout described above. Neither dataset is redistributed in the repository, in line with their CC BY-NC licences; `scripts/download_data.sh` retrieves both from their official sources, and the README documents the process.

6 Results and Discussion

6.1 Architecture Comparison

Table 1 reports ten-fold cross-validated performance for the three configurations.

Table 1: Ten-fold cross-validation results on UrbanSound8K (mean $\pm$ standard deviation across folds).

Model	Parameters	Accuracy (%)	Macro F1
MLP baseline	5,804,810	54.61 $\pm$ 5.13	0.552 $\pm$ 0.053
CNN (no augmentation)	390,890	75.39 $\pm$ 5.02	0.767 $\pm$ 0.044
CNN + augmentation	390,890	75.07 $\pm$ 7.01	0.761 $\pm$ 0.062

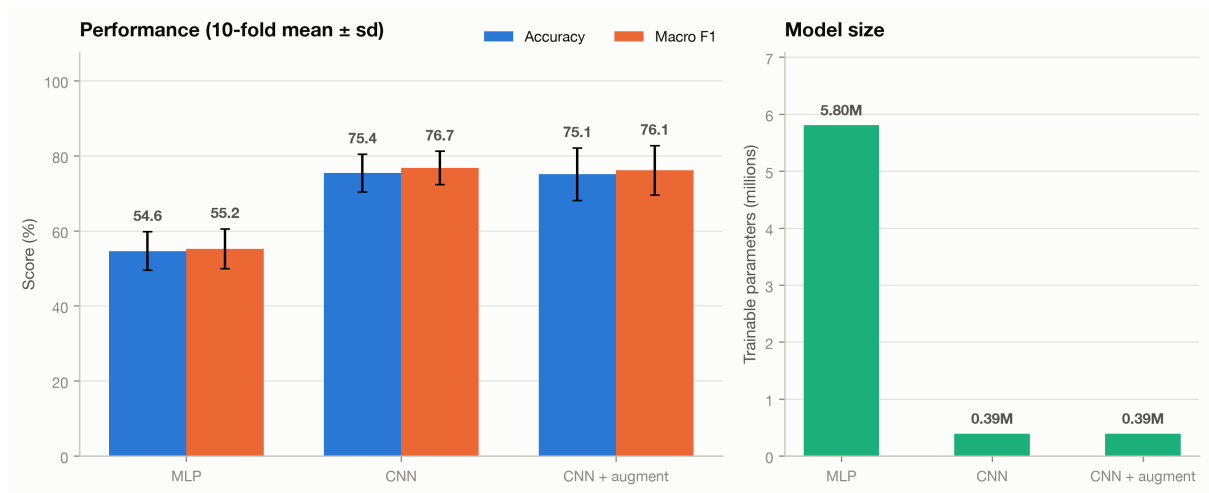


Figure 4: Performance and model size side by side. The CNN achieves higher accuracy and macro F1 than the MLP while using roughly fifteen times fewer parameters.

The central finding is that the convolutional network outperforms the perceptron by 20.8 percentage points while using roughly fifteen times fewer parameters. Since the MLP has strictly greater capacity, the gap cannot be model size; it is architectural inductive bias. Convolution encodes two assumptions that hold for spectrograms and that the MLP must otherwise learn from scratch: informative structure is *local* in time and frequency, and the same pattern is meaningful wherever it appears. Weight sharing learns a harmonic-onset filter once and applies it everywhere, whereas the MLP must relearn that pattern at every position — from only a few thousand examples.

Augmentation, by contrast, did *not* help: the augmented CNN scored 75.07% against 75.39% without it, a statistically indistinguishable difference with slightly higher fold variance (± 7.01 versus ± 5.02). Reported as the negative result it is, the likely cause is that the architecture is already strongly regularised — dropout, weight decay, label smoothing, and global average pooling — so the validation curves in Figure 5 show only mild overfitting to begin with. With little overfitting to recover, SpecAugment adds input noise rather than signal. This matches the literature view that augmentation helps most when a model is data-starved and overfitting hard, which is not the regime here. The remaining analysis therefore uses the unaugmented CNN.

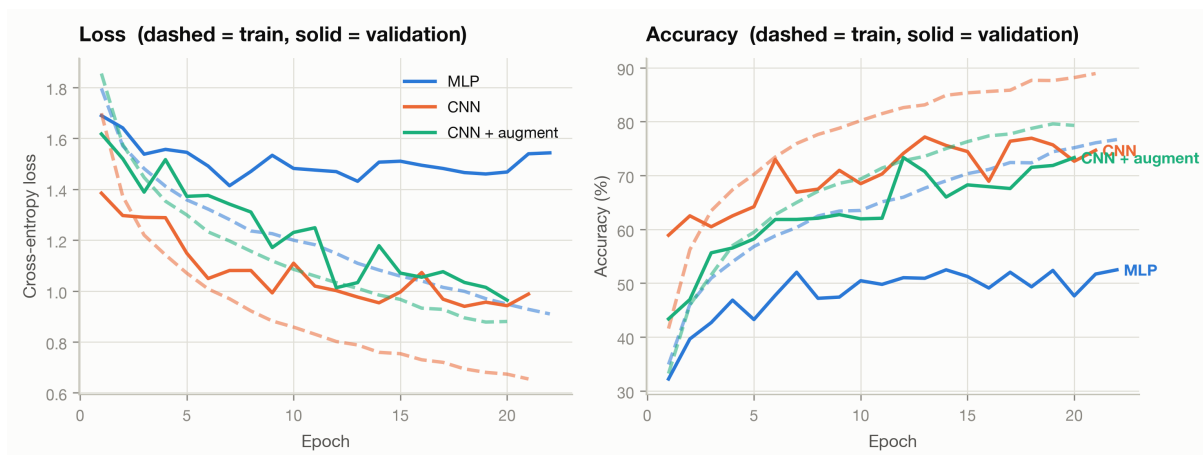


Figure 5: Training and validation curves on a representative fold. Dashed lines are training, solid lines validation. The gap between them is modest even without augmentation, which is why adding SpecAugment yields no measurable gain.

6.2 Per-Class Performance and Error Structure

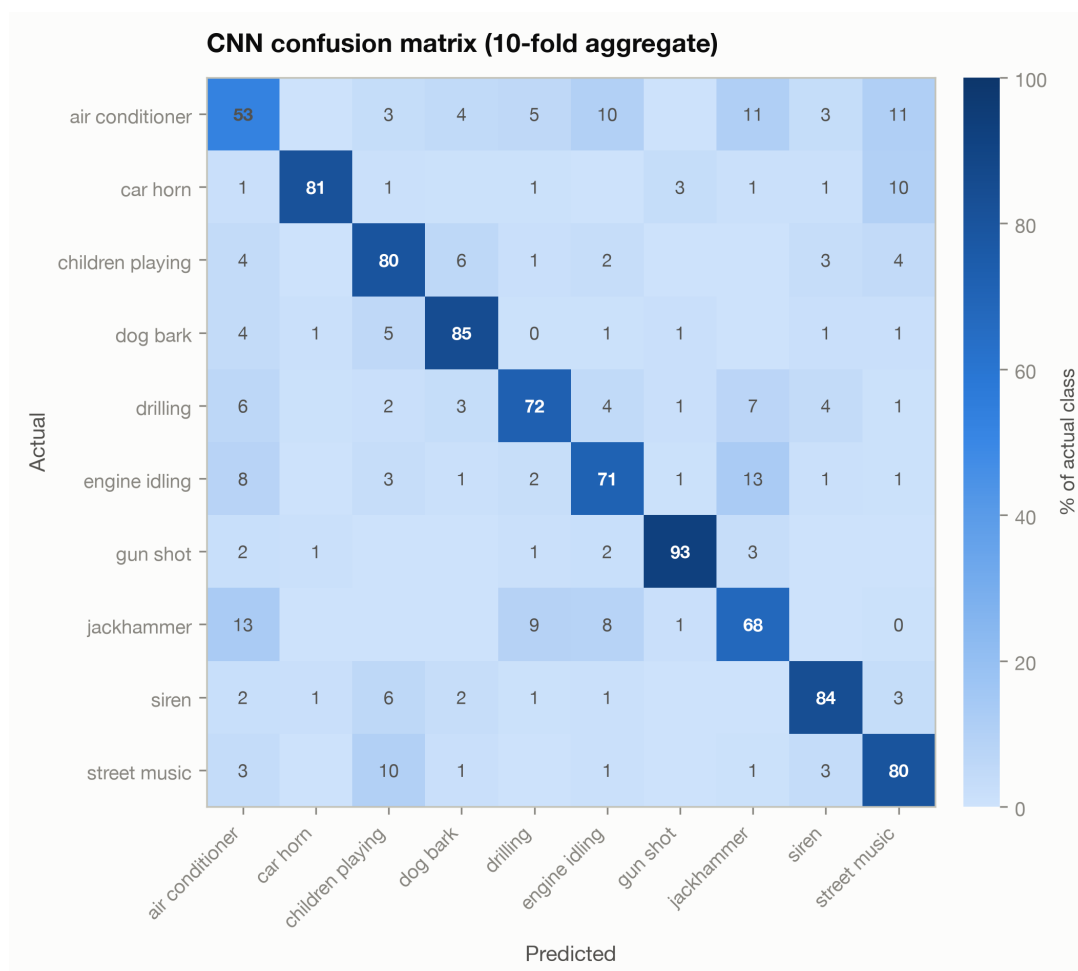


Figure 6: Aggregate confusion matrix over all ten held-out folds, normalised per row. Each clip is predicted exactly once, when its fold is the test fold.

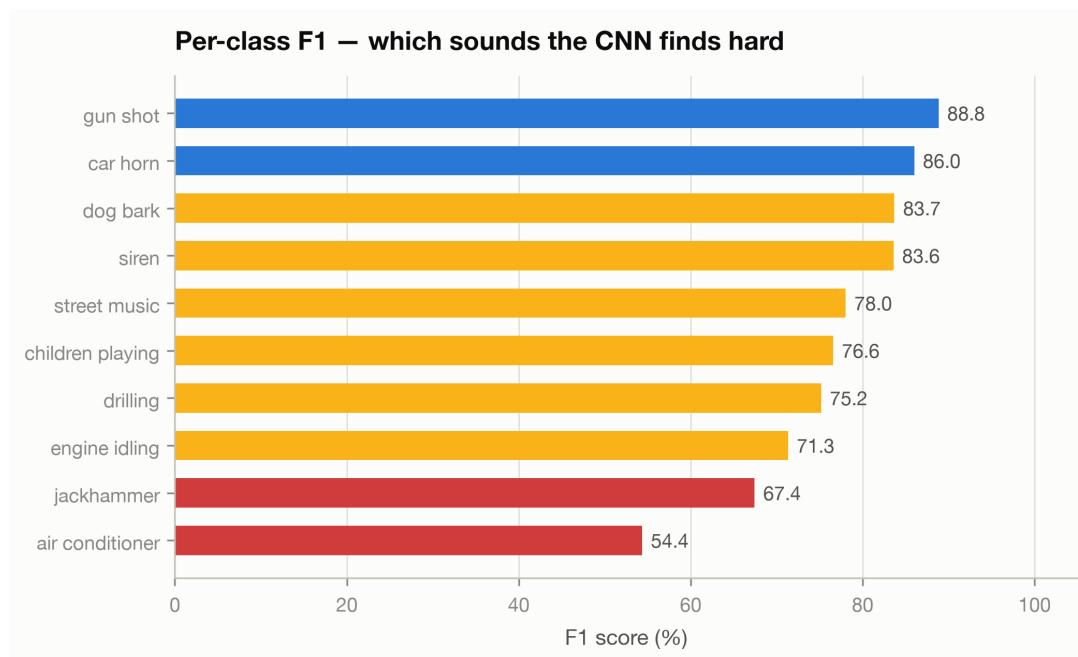


Figure 7: Per-class F1. Impulsive, spectrally distinctive sounds are recognised most reliably; sustained broadband machine noise is the principal source of error.

The strongest class is `gun_shot` ($F1 = 0.888$) and the weakest `air_conditioner` ($F1 = 0.544$). The error structure in Figure 6 is acoustically interpretable rather than arbitrary: the dominant confusions occur among `air_conditioner`, `engine_idling`, `drilling`, and `jackhammer`. All four are sustained, broadband, low-frequency machine noises whose log-mel representations genuinely resemble one another, and human listeners confuse them under similar conditions. By contrast, classes with sharp temporal or harmonic signatures — `gun_shot`, `siren`, `dog_bark` — are well separated.

This distinction matters for the intended application. The four safety-critical classes are drawn from the well-separated group, so the errors the system does make are concentrated in the classes where a mistake carries the least consequence: confusing an air conditioner with an idling engine has no safety implication, whereas missing a siren would.

6.3 Clustering the Learned Representation

To examine what the network learned independently of its classification head, 256-dimensional embeddings were extracted from the held-out fold — data the network never trained on — and clustered with k-means ($k = 10$) without any label information.

Table 2: Agreement between unsupervised clusters and true classes.

Metric	Value
Adjusted Rand Index	0.426
Normalised Mutual Information	0.577
Silhouette coefficient	0.286

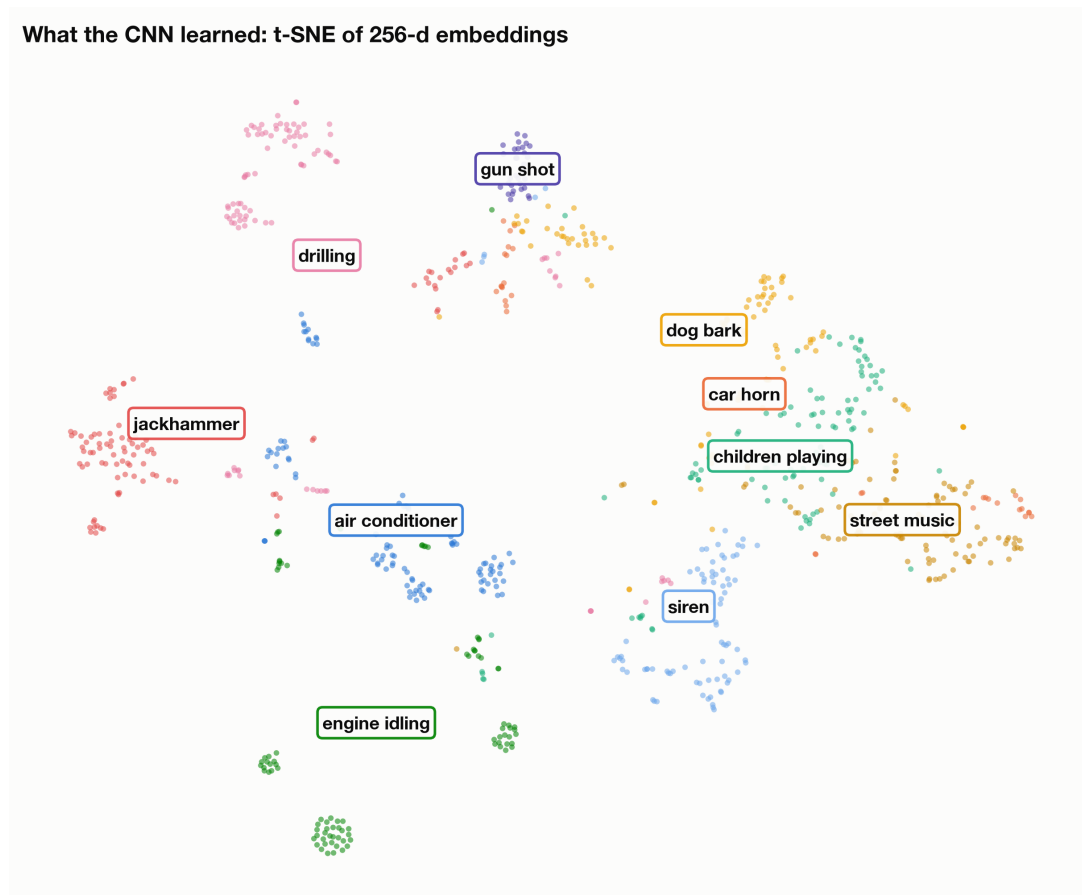


Figure 8: t-SNE projection of the CNN’s 256-dimensional embeddings for held-out clips, with each cluster labelled directly at its centroid. Classes form coherent groups without the clustering algorithm being given any labels.

That k-means recovers the class structure to this degree confirms the network learned a genuinely organised representation rather than an arbitrary decision boundary. The t-SNE projection is also diagnostically useful: the classes that overlap in embedding space are the same ones that the confusion matrix shows being misclassified, so the two analyses corroborate each other. The machine-noise classes form a single contiguous region, which explains at a representational level why they are difficult to separate.

6.4 Novel-Sound Detection

The classifier was then exposed to the eight unseen ESC-50 classes. Table 3 compares the autoencoder against the softmax-confidence baseline.

Table 3: Novelty detection: distinguishing unseen ESC-50 sounds from known UrbanSound8K sounds.

Method	ROC-AUC	Average precision
Softmax confidence baseline [6]	0.703	—
Convolutional autoencoder	0.760	0.613

At the 95th-percentile threshold the autoencoder achieves precision 0.714, recall 0.328, and F1 0.450.

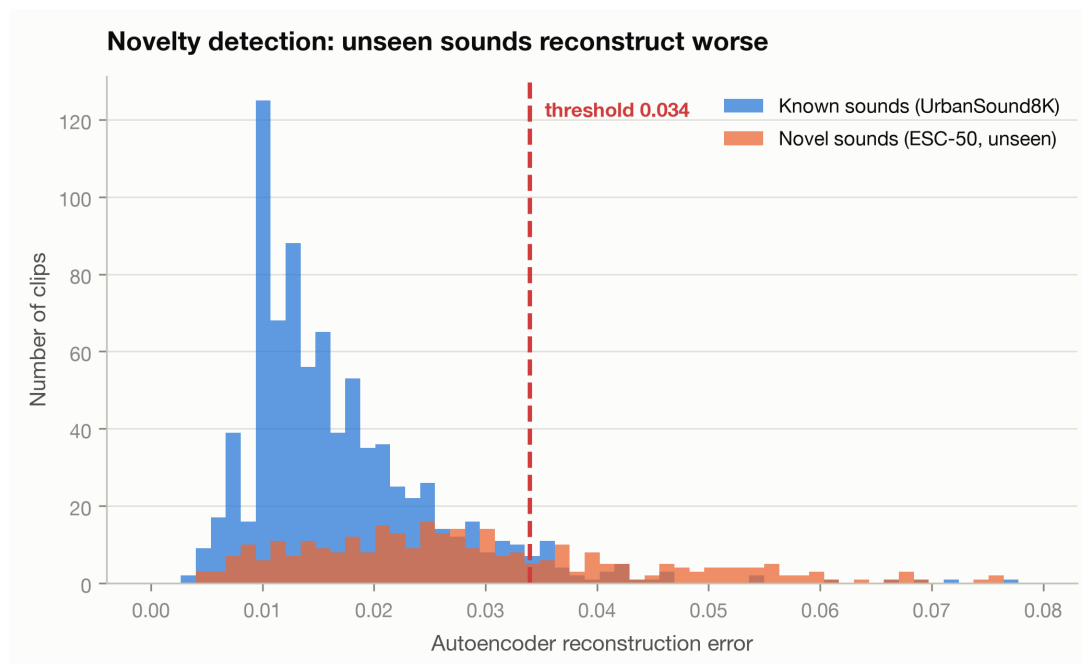


Figure 9: Distribution of reconstruction error for known and unseen sounds, with the operating threshold marked. Separation between the distributions is what makes the score usable.

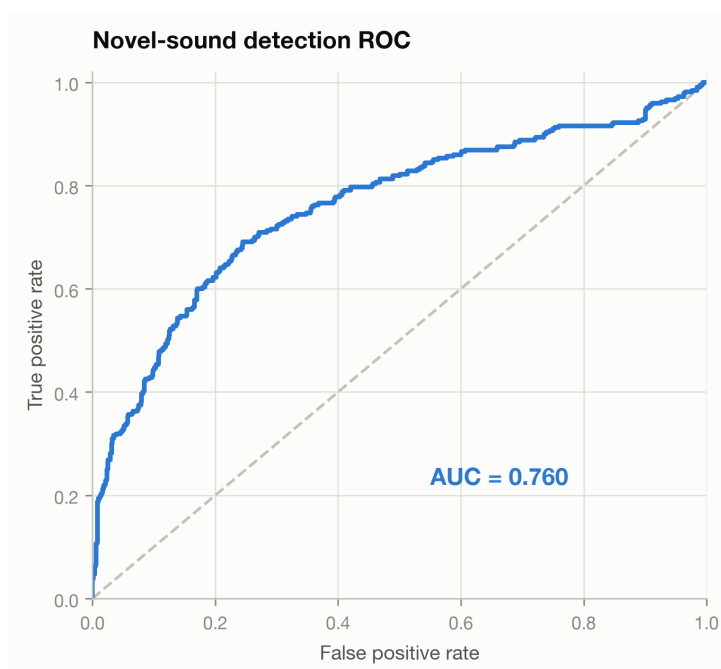


Figure 10: ROC curve for novel-sound detection using autoencoder reconstruction error.

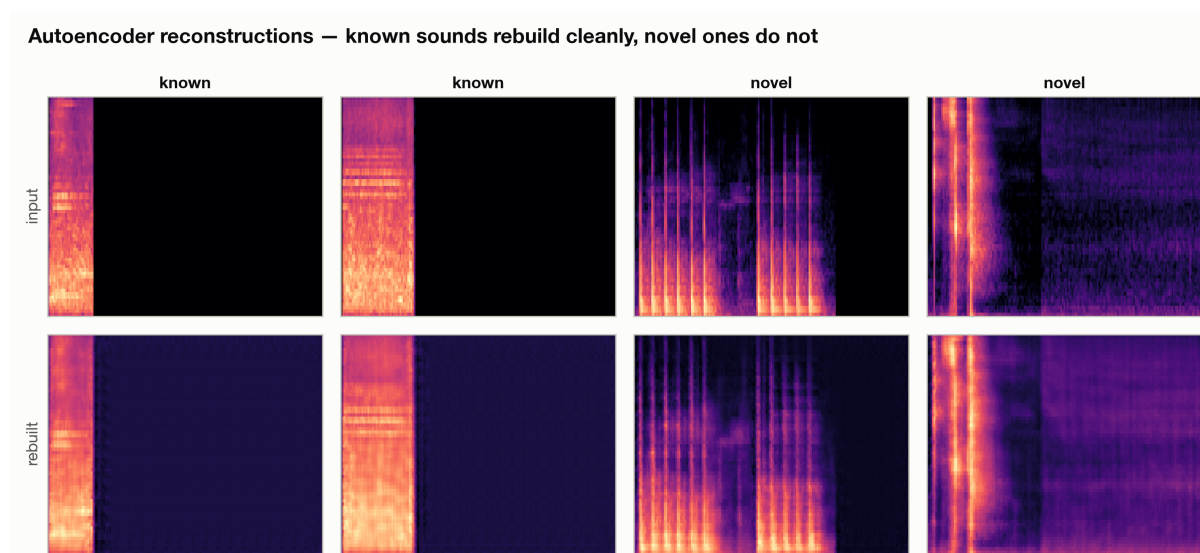


Figure 11: Autoencoder input and reconstruction for known and unseen sounds. Known sounds are rebuilt with their structure intact; unseen sounds lose detail, and that loss is the detection signal.

The key finding is the classifier’s confidence on sounds it has never encountered: mean maximum softmax probability on the unseen ESC-50 clips was 0.530. A closed-set classifier does not merely err on unfamiliar input — it errs *confidently*, because softmax normalises over the ten known classes and cannot express “none of these”. For an assistive device this is the most dangerous failure mode, since a confidently wrong label is more likely to be acted upon. The autoencoder addresses exactly this gap, and Table 3 quantifies what it adds over the cheaper softmax heuristic.

6.5 Real-Time Demonstration

The trained models are deployed in `demo/live_demo.py`, which keeps a rolling four-second microphone buffer, recomputes the spectrogram on the training front-end, and runs both networks twice per second. The interface shows the live spectrogram, per-class confidence, and a status banner with three states: an ordinary recognised sound, a safety-critical one, and a sound the autoencoder flags as unfamiliar. Predictions are smoothed with an exponential moving average to prevent single-frame flicker, and because the models are only 390,890 and 28,321 parameters, both run in real time on laptop CPU or MPS (20–30 ms per window). A recorded walkthrough is linked in Section 5.8.

7 Limitations and Future Work

- **Small vocabulary.** UrbanSound8K’s taxonomy is urban and outdoor-oriented; a deployed device would need the household sounds that matter most (doorbell, smoke alarm, name being called), requiring a combined taxonomy or purpose-built data collection.
- **Clean recordings, noisy world.** Both datasets contain isolated events, but real environments layer sounds continuously, and a single-label classifier cannot express “a siren *and* street music”. Multi-label sound event detection over time is the correct formulation.
- **Fixed four-second windows.** A gunshot is padded with silence; a continuous jackhammer is truncated. Variable-length modelling or attention pooling would fit the data better.

- **Coarse novelty detection.** Reconstruction error flags that an input is unfamiliar but not what it is, and the threshold trades false alarms against misses along one axis; per-class calibration would likely be needed in production.
- **Single-device evaluation.** Robustness across microphones, room acoustics, and distances — all of which shift the spectrogram distribution — was not measured. A transfer-learning baseline (fine-tuning an AudioSet-pretrained network) is the obvious next experiment.

8 Social, Ethical, Legal, and Professional Considerations

8.1 Privacy

An always-listening microphone is among the most privacy-sensitive components in a home. The design mitigates this: all inference is local, the four-second buffer is overwritten continuously, and no audio is ever recorded, transmitted, or stored. A deployed version must preserve this, since a cloud implementation would create a permanent record of domestic conversation as a side effect of listening for a doorbell.

8.2 Asymmetric Consequences of Error

The two error types are not equivalent: a false alarm is a minor irritation, whereas a missed siren or smoke alarm has real safety consequences. This argues for tuning the safety-critical classes toward recall at the cost of precision — the opposite of what maximising overall accuracy suggests — and for reporting per-class metrics rather than a single headline number.

8.3 Bias and Dataset Provenance

Both datasets are drawn from Freesound and skewed toward Europe and North America. Sirens especially differ between countries, and horns, alarms, and street noise vary regionally, so this system should not be assumed to transfer to Kathmandu without local evaluation and, most likely, local data collection. Presenting it as universally applicable would be misleading.

8.4 Framing, Accessibility, and Licensing

The system is a supplementary awareness aid, not a safety-certified alerting device, and must not be positioned as a replacement for certified accessible smoke alarms; genuine accessibility work would also require participation from Deaf users rather than assumptions made on their behalf. Finally, both datasets are Creative Commons Attribution Non-Commercial: their use here is consistent with those terms, both are cited, and neither is redistributed — the download script retrieves them from their official sources.

9 Conclusion

This project built and evaluated a neural network system for recognising urban and household sounds, framed around assistive awareness for Deaf and hard-of-hearing users. Four tasks were performed under UrbanSound8K’s official ten-fold protocol: classification with a convolutional network on log-mel spectrograms, novelty detection with a convolutional autoencoder, unsupervised clustering of the learned embeddings, and an architecture comparison with an augmentation ablation.

The convolutional network reached 75.39% accuracy and 0.767 macro F1 using 390,890 parameters, outperforming the larger 5,804,810-parameter perceptron by 20.8 points. Because the baseline is the larger model, this isolates the contribution of convolutional inductive bias

rather than capacity. Augmentation did not improve on it — a negative result reported honestly and attributed to the architecture already being well-regularised. The autoencoder detected unseen sounds with ROC-AUC 0.760, ahead of the 0.703 softmax baseline.

Two observations carry forward. First, the classifier’s errors are acoustically structured, not random: confusion concentrates among sustained broadband machine noises, while the safety-critical classes sit in the well-separated region of the embedding space — shown independently by the confusion matrix and the t-SNE projection. Second, the closed-set classifier is confident on inputs it has never seen (mean softmax 0.530), which a fixed-test-set accuracy never reveals; any open-environment system needs an explicit way to signal that an input falls outside its competence.

Most concretely, the project demonstrated the capability that distinguishes neural networks from the classical methods covered elsewhere in the programme: no acoustic feature was designed by hand. Given only a time–frequency representation, the network learned the filters that separate a siren from a jackhammer — and the clustering confirms that representation is structured enough to recover the classes without labels at all.

References

References

- [1] J. Salamon, C. Jacoby, and J. P. Bello, “A dataset and taxonomy for urban sound research,” *Proc. 22nd ACM International Conference on Multimedia*, pp. 1041–1044, 2014.
- [2] K. J. Piczak, “ESC: Dataset for environmental sound classification,” *Proc. 23rd ACM International Conference on Multimedia*, pp. 1015–1018, 2015.
- [3] K. J. Piczak, “Environmental sound classification with convolutional neural networks,” *Proc. IEEE International Workshop on Machine Learning for Signal Processing (MLSP)*, pp. 1–6, 2015.
- [4] J. Salamon and J. P. Bello, “Deep convolutional neural networks and data augmentation for environmental sound classification,” *IEEE Signal Processing Letters*, vol. 24, no. 3, pp. 279–283, 2017.
- [5] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le, “SpecAugment: A simple data augmentation method for automatic speech recognition,” *Proc. Interspeech*, pp. 2613–2617, 2019.
- [6] D. Hendrycks and K. Gimpel, “A baseline for detecting misclassified and out-of-distribution examples in neural networks,” *Proc. International Conference on Learning Representations (ICLR)*, 2017.
- [7] M. Sakurada and T. Yairi, “Anomaly detection using autoencoders with nonlinear dimensionality reduction,” *Proc. MLSDA 2nd Workshop on Machine Learning for Sensory Data Analysis*, pp. 4–11, 2014.
- [8] M. Huzaifah, “Comparison of time-frequency representations for environmental sound classification using convolutional neural networks,” *arXiv preprint arXiv:1706.07156*, 2017.
- [9] B. McFee, C. Raffel, D. Liang, D. P. W. Ellis, M. McVicar, E. Battenberg, and O. Nieto, “librosa: Audio and music signal analysis in Python,” *Proc. 14th Python in Science Conference*, pp. 18–25, 2015.

- [10] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” *Proc. 32nd International Conference on Machine Learning (ICML)*, pp. 448–456, 2015.
- [11] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [12] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” *Proc. International Conference on Learning Representations (ICLR)*, 2019.
- [13] M. Lin, Q. Chen, and S. Yan, “Network in network,” *Proc. International Conference on Learning Representations (ICLR)*, 2014.
- [14] L. van der Maaten and G. Hinton, “Visualizing data using t-SNE,” *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.
- [15] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, pp. 8026–8037, 2019.

A Submission Resources and Links

- **GitHub repository** (public):
<https://github.com/SudipAdh/acoustic-scene-awareness>
- **Dataset — UrbanSound8K** (primary, 8,732 clips, 10 classes):
<https://urbansounddataset.weebly.com/urbansound8k.html>
- **Dataset — ESC-50** (unseen sounds for novelty detection, 2,000 clips):
<https://github.com/karolpiczak/ESC-50>
- **YouTube demonstration video:**
[YOUTUBE DEMO LINK TO BE INSERTED]

The repository is public and contains the complete source code, the LaTeX sources for this report, all generated figures, the per-experiment result files, the Colab notebook, and the trained model weights. Neither dataset is redistributed in the repository, in line with their CC BY-NC licences; `scripts/download_data.sh` retrieves both from the official sources above.

B Project Proposal (As Submitted)

[Paste the approved project proposal here, exactly as submitted, per the assignment brief.]

C Reproduction Instructions

```
# 1. Environment (Python 3.11)
uv venv --python 3.11 --python-preference only-managed
uv pip install -r requirements.txt

# 2. Datasets (~6.5 GB, downloads from official sources)
bash scripts/download_data.sh

# 3. Fast sanity check
.venv/bin/python run_experiments.py --quick

# 4. Full experiment suite
.venv/bin/python run_experiments.py

# 5. Live demonstration
.venv/bin/python demo/live_demo.py
```

D Experiment Evidence

The staged runner (`run_experiments.py`) executes the full pipeline — feature preparation, the MLP-versus-CNN comparison, the final CNN evaluation, embedding clustering, and autoencoder novelty detection — on the development machine. The captures below are from a fast verification run (one fold, three epochs) that exercises every stage end to end; the environment banner confirms the Apple M1 host and the Metal (mps) device. The headline accuracies reported in Section 6 come from the full 10-fold, 40-epoch protocol rather than this short run, so the per-fold numbers shown here are intentionally lower.

```
python run_experiments.py -- compare stage (MLP vs CNN)

>>> QUICK MODE: 1 fold, 3 epochs

=====
ACOUSTIC SCENE AWARENESS -- EXPERIMENT RUN
=====

host      : Sudips-MacBook-Pro-M1.local
platform  : macOS-26.2-arm64-arm-64bit
machine   : arm64 (arm)
python    : 3.11.15
torch     : 2.13.0
device    : mps
folds     : [1]
epochs    : 3

STAGE: compare -- MLP vs CNN, and augmentation ablation
=====

=====
MLP | folds=[1] augment=False
=====
fold1 mlp ep 1/3 train 1.796/0.349 val 1.690/0.322
fold1 mlp -> test acc 0.5441 f1 0.5465 (5s)

MLP SUMMARY acc 0.5441 +/- 0.0000 | macro-F1 0.5465 +/- 0.0000 | params 5,804,810
saved -> /Users/sudipadh/Desktop/PERSONAL/masters/ANN/acoustic-scene-awareness/results/logs/cv_mlp.json

=====
CNN | folds=[1] augment=False
=====
fold1 cnn ep 1/3 train 1.699/0.416 val 1.385/0.589
fold1 cnn -> test acc 0.7743 f1 0.7704 (37s)

CNN SUMMARY acc 0.7743 +/- 0.0000 | macro-F1 0.7704 +/- 0.0000 | params 390,890
saved -> /Users/sudipadh/Desktop/PERSONAL/masters/ANN/acoustic-scene-awareness/results/logs/cv_cnn_noaug.json

=====
CNN | folds=[1] augment=True
=====
fold1 cnn ep 1/3 train 1.855/0.332 val 1.617/0.434
fold1 cnn -> test acc 0.6472 f1 0.6322 (39s)

CNN SUMMARY acc 0.6472 +/- 0.0000 | macro-F1 0.6322 +/- 0.0000 | params 390,890
saved -> /Users/sudipadh/Desktop/PERSONAL/masters/ANN/acoustic-scene-awareness/results/logs/cv_cnn_aug.json
figure -> /Users/sudipadh/Desktop/PERSONAL/masters/ANN/acoustic-scene-awareness/results/figures/fig_model_comparison.png
figure -> /Users/sudipadh/Desktop/PERSONAL/masters/ANN/acoustic-scene-awareness/results/figures/fig_training_curves.png
saved -> /Users/sudipadh/Desktop/PERSONAL/masters/ANN/acoustic-scene-awareness/results/logs/comparison_summary.json
```

Figure 12: Environment banner and the architecture-comparison stage running on the Apple M1 (device: mps). The MLP and CNN are trained and scored on the official fold split.

```
python run_experiments.py - final CNN evaluation

STAGE: final - full CNN evaluation
=====
fold1 cnn ep 1/3 train 1.855/0.332 val 1.617/0.434
fold1 cnn -> test acc 0.6472 f1 0.6322 (39s)

      precision    recall  f1-score   support

air_conditioner    0.530    0.710    0.607       100
car_horn           0.647    0.611    0.629        36
children_playing   0.831    0.690    0.754       100
dog_bark           0.783    0.650    0.710       100
drilling           0.561    0.780    0.653       100
engine_idling      0.542    0.271    0.361        96
gun_shot           0.408    0.571    0.476        35
jackhammer         0.863    0.575    0.690       120
siren              0.787    0.860    0.822        86
street_music       0.550    0.710    0.620       100

      accuracy          0.647       873
macro avg           0.650    0.643    0.632       873
weighted avg        0.672    0.647    0.644       873

Accuracy 0.6472 | macro-F1 0.6322
figure -> /Users/sudipadh/Desktop/PERSONAL/masters/ANN/acoustic-scene-awareness/results/figures/fig_confusion_matrix.png
figure -> /Users/sudipadh/Desktop/PERSONAL/masters/ANN/acoustic-scene-awareness/results/figures/fig_per_class_f1.png
saved -> /Users/sudipadh/Desktop/PERSONAL/masters/ANN/acoustic-scene-awareness/results/logs/final_metrics.json
```

Figure 13: The final-evaluation stage emitting the per-class precision, recall, and F1 report, then writing the confusion-matrix and per-class figures used in the report.

```
python run_experiments.py - clustering + novelty detection

STAGE: cluster - k-means + t-SNE on CNN embeddings
=====
embeddings: (837, 256)
ARI 0.275 | NMI 0.447 | silhouette 0.289
figure -> /Users/sudipadh/Desktop/PERSONAL/masters/ANN/acoustic-scene-awareness/results/figures/fig_tsne_embeddings.png
saved -> /Users/sudipadh/Desktop/PERSONAL/masters/ANN/acoustic-scene-awareness/results/logs/cluster_metrics.json

=====
STAGE: anomaly - autoencoder novelty detection
=====
novel (unseen) clips: (320, 64, 173)
AE ep 1/3 train 0.18375 val 0.06433
AE trained in 12s
AE novelty ROC-AUC 0.808 | AP 0.660 | P 0.729 R 0.353 F1 0.476
softmax baseline ROC-AUC 0.540 (mean confidence: known 0.495, novel 0.458)
figure -> /Users/sudipadh/Desktop/PERSONAL/masters/ANN/acoustic-scene-awareness/results/figures/fig_anomaly_scores.png
figure -> /Users/sudipadh/Desktop/PERSONAL/masters/ANN/acoustic-scene-awareness/results/figures/fig_anomaly_roc.png
figure -> /Users/sudipadh/Desktop/PERSONAL/masters/ANN/acoustic-scene-awareness/results/figures/fig_ae_reconstructions.png
saved -> /Users/sudipadh/Desktop/PERSONAL/masters/ANN/acoustic-scene-awareness/results/logs/anomaly_results.json

=====
DONE in 5.7 min - figures in /Users/sudipadh/Desktop/PERSONAL/masters/ANN/acoustic-scene-awareness/results/figures
=====
```

Figure 14: The clustering and novelty-detection stages: t-SNE/k-means metrics on the learned embeddings, followed by the autoencoder novelty ROC-AUC against the softmax-confidence baseline.

E Key Implementation Code

The complete, runnable source code is public at <https://github.com/SudipAdh/acoustic-scene-awareness>, the entire codebase is therefore not reproduced here. The listings below capture the sections most important to understanding the implementation, and the five core excerpts (the log-mel front-end, the official fold split, the CNN forward pass, the novelty score, and the threshold calibration) appear inline in Section 5. Screen captures of these sections in the development

environment follow.

```
src/data.py – log-mel spectrogram front-end

56 def audio_to_melspec(y: np.ndarray, sr: int = C.SAMPLE_RATE) -> np.ndarray:
57     """Convert a waveform to a log-scaled mel-spectrogram.
58
59     This is the core "audio becomes an image" step: the result is a
60     (N_MELS, N_FRAMES) matrix of energy per frequency band over time, which a
61     2-D CNN can process exactly like a grayscale image.
62
63     Decibel scaling matters – human loudness perception is logarithmic, and
64     without it a handful of loud frames dominate the input range.
65     """
66     import librosa
67
68     mel = librosa.feature.melspectrogram(
69         y=y,
70         sr=sr,
71         n_fft=C.N_FFT,
72         hop_length=C.HOP_LENGTH,
73         n_mels=C.N_MELS,
74         fmin=C.F_MIN,
75         fmax=C.F_MAX,
76     )
77     mel_db = librosa.power_to_db(mel, ref=np.max)
78     return mel_db.astype(np.float32)
```

Figure 15: The log-mel front-end (`src/data.py`): the “audio becomes an image” step that turns each clip into a decibel-scaled time–frequency matrix the CNN consumes like a grayscale image.

```
src/data.py – official UrbanSound8K fold split

209 def make_fold_split(X, meta, test_fold: int, val_fold: int | None = None):
210     """Split by UrbanSound8K's official folds.
211
212     Clips sliced from the same source recording share a fold, so splitting by
213     fold (rather than randomly) is what keeps the evaluation honest.
214     """
215     if val_fold is None:
216         val_fold = test_fold % C.N_FOLDS + 1
217
218     folds = meta["fold"].values
219     labels = meta["classID"].values
220
221     te = folds == test_fold
222     va = folds == val_fold
223     tr = ~(te | va)
224
225     return (X[tr], labels[tr]), (X[va], labels[va]), (X[te], labels[te])
```

Figure 16: The official UrbanSound8K fold split (`src/data.py`), which keeps every clip in exactly one test fold and prevents the recording-level leakage that inflates naive random splits.

```

src/models.py – AudioCNN (convolutional classifier)

76 class AudioCNN(nn.Module):
77     """Four-block CNN over log-mel spectrograms.
78
79     Early filters learn local time-frequency texture (onsets, harmonic stacks,
80     broadband noise); deeper blocks compose those into class-level patterns.
81
82     Global average pooling replaces a large flatten+dense head. That keeps the
83     parameter count ~10x below the MLP and makes the model robust to a sound
84     occurring at a different point in the clip.
85
86     `return_embedding=True` exposes the 256-d penultimate features, which the
87     clustering / t-SNE analysis uses to show what the network learned.
88     """
89
90     def __init__(self, n_classes: int = 10, dropout: float = 0.3, width: int = 32):
91         super().__init__()
92         w = width
93         self.block1 = ConvBlock(1, w, pool=2)
94         self.block2 = ConvBlock(w, w * 2, pool=2, dropout=dropout / 2)
95         self.block3 = ConvBlock(w * 2, w * 4, pool=2, dropout=dropout / 2)
96         self.block4 = ConvBlock(w * 4, w * 8, pool=2, dropout=dropout)
97
98         self.gap = nn.AdaptiveAvgPool2d(1)
99         self.dropout = nn.Dropout(dropout)
100         self.fc = nn.Linear(w * 8, n_classes)
101         self.embed_dim = w * 8
102
103     def forward(self, x, return_embedding: bool = False):
104         x = self.block1(x)
105         x = self.block2(x)
106         x = self.block3(x)
107         x = self.block4(x)
108
109         emb = self.gap(x).flatten(1)
110         logits = self.fc(self.dropout(emb))
111
112         if return_embedding:
113             return logits, emb
114         return logits
115
116
117 # -----
118 # Novelty detection: convolutional autoencoder

```

Figure 17: The convolutional classifier (`src/models.py`): four `ConvBlocks` followed by global average pooling, with `return_embedding` exposing the 256-d features used for clustering.

```

src/models.py – ConvAutoencoder (novelty detection)

120 class ConvAutoencoder(nn.Module):
121     """Compresses a spectrogram to a small latent code and rebuilds it.
122
123     Trained only on the ten known UrbanSound8K classes, it learns to reconstruct
124     those well. A sound it has never encountered (a smashing window, a crying
125     baby) falls outside that learned manifold and reconstructs poorly, so the
126     per-clip reconstruction error works directly as a novelty score.
127     """
128
129     def __init__(self, latent_ch: int = 32):
130         super().__init__()
131
132         self.encoder = nn.Sequential(
133             nn.Conv2d(1, 16, 3, stride=2, padding=1), # -> 32 x 87
134             nn.BatchNorm2d(16),
135             nn.ReLU(),
136             nn.Conv2d(16, 32, 3, stride=2, padding=1), # -> 16 x 44
137             nn.BatchNorm2d(32),
138             nn.ReLU(),
139             nn.Conv2d(32, latent_ch, 3, stride=2, padding=1), # -> 8 x 22
140             nn.BatchNorm2d(latent_ch),
141             nn.ReLU(),
142         )
143
144         self.decoder = nn.Sequential(
145             nn.ConvTranspose2d(latent_ch, 32, 3, stride=2, padding=1, output_padding=1),
146             nn.BatchNorm2d(32),
147             nn.ReLU(),
148             nn.ConvTranspose2d(32, 16, 3, stride=2, padding=1, output_padding=1),
149             nn.BatchNorm2d(16),
150             nn.ReLU(),
151             nn.ConvTranspose2d(16, 1, 3, stride=2, padding=1, output_padding=1),
152         )
153
154     def forward(self, x):
155         z = self.encoder(x)
156         out = self.decoder(z)
157         # Transposed convolutions can overshoot by a pixel; crop to match input.
158         return out[..., : x.shape[-2], : x.shape[-1]]
159
160     def encode(self, x):
161         return self.encoder(x)
162
163     @torch.no_grad()
164     def reconstruction_error(self, x) -> torch.Tensor:
165         """Per-sample mean squared error – the novelty score."""
166         recon = self(x)
167         return ((recon - x) ** 2).flatten(1).mean(dim=1)
168
169
170 # -----
171 # Helpers

```

Figure 18: The convolutional autoencoder (`src/models.py`) used for novelty detection; `reconstruction_error` yields the per-clip score that flags out-of-distribution sounds.

```

src/train.py – 10-fold cross-validation loop

191 def cross_validate(
192     X,
193     meta,
194     model_name: str = "cnn",
195     folds=None,
196     epochs: int = C.EPOCHS,
197     augment: bool = True,
198     tag: str | None = None,
199     **kwargs,
200 ):
201     """Run the official 10-fold protocol and report mean +/- std.
202
203     Two conveniences for long unattended runs, both controlled by environment
204     variables so they never affect the results:
205
206     * ``COOLDOWN_SEC`` pauses between folds to let the GPU shed heat. Folds are
207     independent, so this changes timing only, never the numbers.
208     * ``RESUME=1`` skips a configuration whose cached JSON already covers the
209     requested folds, so a restarted run does not redo finished work. Results
210     are deterministic, so a skipped config is identical to a rerun.
211     """
212     import os
213
214     folds = folds or list(range(1, C.N_FOLDS + 1))
215     cooldown = float(os.environ.get("COOLDOWN_SEC", "0"))
216
217     name = tag or f"{model_name}_aug{int(augment)}"
218     out = C.LOGS_DIR / f"cv_{name}.json"
219
220     if os.environ.get("RESUME") == "1" and out.exists():
221         cached = json.loads(out.read_text())
222         if set(cached.get("folds", [])) >= set(folds):
223             print(f"[resume] {name} already complete "
224                   f"({cached['acc_mean']*100:.2f}%) -- skipping")
225             return cached
226
227     results: list[FoldResult] = []
228
229     print(f"\n{' '*70}\n{model_name.upper()} | folds={folds} augment={augment}"
230           f"\n{'f' cooldown={cooldown:.0f}s' if cooldown else ''}\n{' '*70}")
231
232     for i, f in enumerate(folds):
233         _, res = train_one_fold(
234             X, meta, test_fold=f, model_name=model_name,
235             epochs=epochs, augment=augment, **kwargs
236         )
237         results.append(res)
238
239         if cooldown and i < len(folds) - 1:
240             print(f" [cooldown] pausing {cooldown:.0f}s to let the GPU cool "
241                   f"(fold {i+1}/{len(folds)} done)")
242             time.sleep(cooldown)

```

Figure 19: The 10-fold cross-validation loop (`src/train.py`), including the environment-controlled cooldown and resume conveniences for long unattended runs.